# Sai Kumar Studio — Project Design & Handover

Oct 23, 2026 · @Chitrada Mohan Vamsi

## 1. Purpose and overview

Sai Kumar Studio is a live online storefront and booking system for a photography studio. It runs as a static customer website, an admin dashboard and a FastAPI backend on Supabase PostgreSQL. This document is the project design document (PDD) and handover pack for the team taking it over.

The site sells across four business areas, each one a "site page" in the catalog:

| Area | What customers do | Typical items |
|---|---|---|
| Photography | Browse packages and book a shoot | Wedding, event and portrait packages in Standard / Premium / Platinum tiers |
| Gifts | Buy personalised gifts, often uploading a photo | Frames, mugs, pillows, crystal, clocks, mirrors |
| Corporate | Order branded or engraved gifts, often in bulk | Gift sets, awards, office essentials |
| Studio | Order print and edit services | Printing, photo editing, finishing |

Three user roles exist: **customer** (shops and books), **staff** (runs the admin dashboard) and **owner** (staff rights plus access to personal data such as login history and IPs).

The code lives in one public Git repository. Secrets, server IPs, key files and database backups must never be committed.

## 2. Scope and features

The system covers the full customer journey from browsing to delivery, plus the staff tools to run it. It does not include a mobile app, a CMS for page text, or browsing/cart analytics.

## Customer site (root `*.html` + `js/`)

| Feature | Pages | Notes |
|---|---|---|
| Catalog browsing and search | `index`, `photography`, `gifts`, `corporate`, `studio`, `bulk-orders`, `equipment-details` | Loaded from `/api/catalog/{page_slug}`; menus from the site menu service |
| Sign in | Auth modal on every page | Email OTP; no passwords for customers |
| Cart and wishlist | `cart` | Kept in the browser and synced to the server when signed in |
| Personalised products | Product pages | Customer uploads artwork (photo); engraving and personalisation fields per product |
| Bulk / corporate pricing | `bulk-orders`, `corporate` | Quantity tiers (`js/shared/bulk-tiers.js`, `services/pricing.py`) |
| Checkout and payment | `cart` | Razorpay online payment or Cash on Delivery; coupons |
| Photography booking | `booking`, `booking-form`, `booking-confirmation` | Package + add-ons + event schedule; a booking needs a signed-in account |
| My account | `my-account`, `my-orders`, `my-bookings` | Addresses, order tracking, cancel or request cancellation, request address change |
| Contact | `contact-us` | Saved as a contact message for staff |

## Admin dashboard (`admin/`, served at `/admin/`)

Staff manage: catalog (pages, categories, products, media, ordering), homepage and menu, coupons, orders (status, tracking, cancellation and address-change requests), bookings, customers (block/unblock, delete), customer artwork, reviews, enquiries, settings and staff accounts. CSV exports exist for orders and bookings. Owners also see the audit log and login activity.

### Order lifecycle

Orders move forward only: `created` → `payment_pending` → `cod_confirmed` or `paid` → `in_production` → `shipped` → `delivered`. `cancelled` and `refunded` are terminal. Stock is reserved at checkout; unpaid orders older than 30 minutes are cancelled and restocked by a background sweep every 5 minutes.

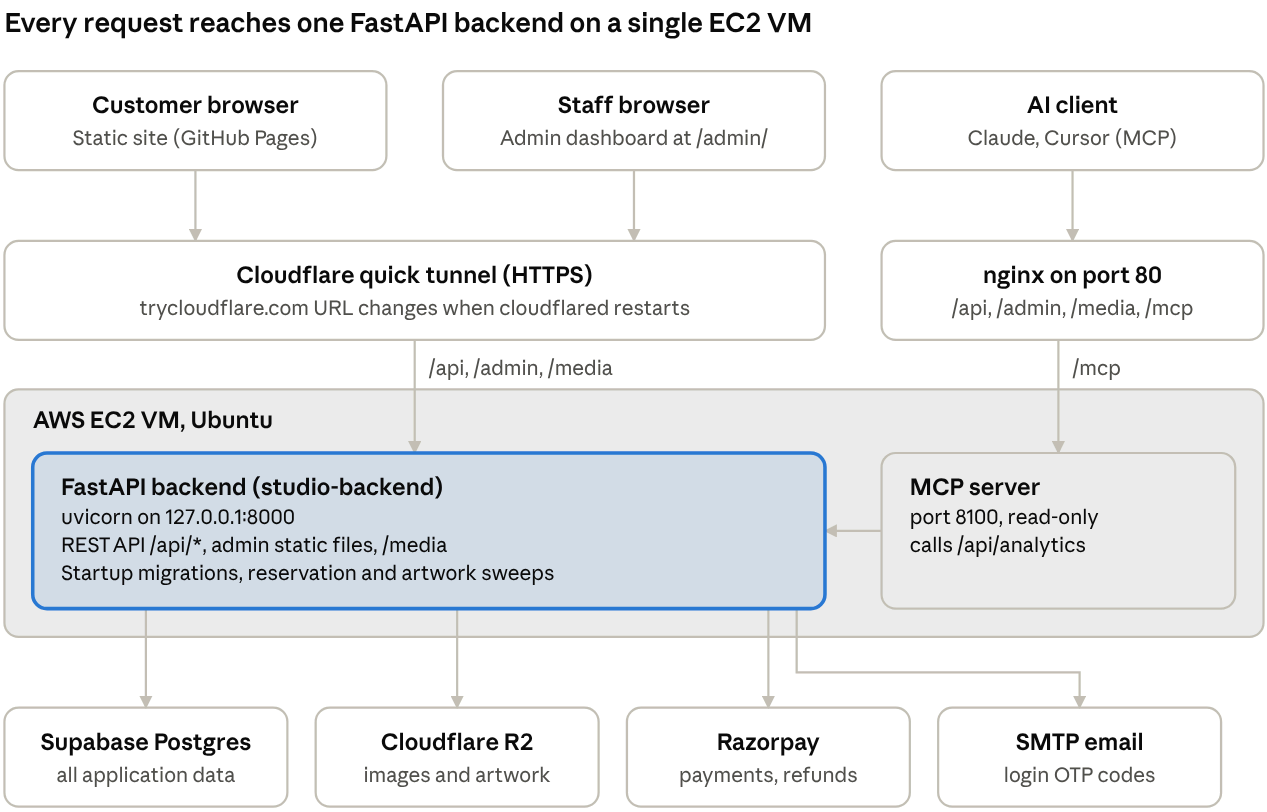
Customers can cancel instantly while `created` / `payment_pending`, and must request cancellation once `cod_confirmed` / `paid`. Bookings start as `enquiry` and are progressed by staff.

### Data retention

Customer artwork is deleted from storage 15 days after delivery, cancellation or refund (owner-configurable via setting `order_artwork_retention_days`). Staff can put any file on hold. Uploads never attached to an order are purged after a short orphan window. The database row is kept with `purged_at` set.

## 3. Architecture

One FastAPI process on a single AWS EC2 VM serves the API, the admin dashboard and media; all data sits in Supabase PostgreSQL. The customer site is plain HTML/JS with no build step and calls the backend over HTTPS.




system architecture · clients, VM, external services

Browsers reach the backend through a free Cloudflare quick tunnel, because the static site is served over HTTPS and cannot call a plain-HTTP IP. nginx on port 80 also proxies to the backend and is the route for the MCP server.

| Layer | Technology | Where |
|---|---|---|
| Customer site | HTML, CSS, vanilla JS (no framework, no build) | Repo root `*.html`, `js/`, `css/`; API base set in `js/shared/api-config.js` |
| Admin dashboard | HTML + vanilla JS modules | `admin/`, served by the backend at `/admin/` |
| Backend | Python 3.12, FastAPI 0.115, SQLAlchemy 2.0, Pydantic 2, uvicorn | `backend/app/` |
| Database | PostgreSQL on Supabase (session pooler) | Connection via `DATABASE_URL` |

| Layer | Technology | Where |
|---|---|---|
| Images | Cloudflare R2 (Cloudinary is legacy, still supported) | `services/storage.py`, `services/media.py` |
| Payments | Razorpay; mock mode when keys are unset | `services/razorpay_service.py`, `routers/payments.py` |
| Analytics for AI tools | MCP server, separate venv | `mcp_server/` |
| Hosting | AWS EC2 Ubuntu, systemd, nginx, cloudflared | `server-config/` |

The app assumes a **single instance**: the reservation sweep, artwork sweep and catalog cache live in-process. Running two copies needs a shared scheduler or lock first.

## 4. Data model

All 25 tables are defined in `backend/app/models.py` with UUID primary keys and `created_at` / `updated_at` timestamps. The catalog is a three-level tree: site page → category → product, with media attached to categories and products.

| Group | Tables | Purpose |
|---|---|---|
| Users and auth | `users`, `otp_codes`, `refresh_tokens`, `blocked_emails` | Accounts with role `customer` / `staff` / `owner`; staff have a password hash, customers sign in by OTP |
| Catalog | `site_pages`, `categories`, `products`, `media`, `reviews` | Four pages; photography tiers are separate product rows; `stock` only for physical products |
| Shopping | `cart_items`, `wishlist_items`, `addresses`, `coupons` | Server copy of cart and wishlist for signed-in users |
| Orders | `orders`, `order_items`, `order_tracking_events`, `order_cancellation_requests`, `order_address_change_requests`, `customer_uploads` | Order with price snapshot, address snapshot, tracking timeline and artwork files |
| Payments | `payments`, `webhook_events` | Razorpay payment records (`created` / `authorized` / `captured` / `failed` / `refunded`); webhook de-duplication |

| Group | Tables | Purpose |
|---|---|---|
| Bookings | `bookings` | Photography bookings with event date, slot, details JSON and advance paid |
| Operations | `notifications`, `contact_messages`, `settings`, `audit_log` | Customer notifications, enquiries, key/value site settings, staff and auth activity |

Key rules:

- Deleting a customer cascades to their orders and bookings; orders block deletion of a user at the database level (`RESTRICT`) unless removed through the admin delete flow.
- Orders store a coupon discount and a bulk (quantity-tier) discount separately; the coupon applies after the bulk discount.
- `audit_log` is the only activity history. It records logins, signups, logouts and admin create/update/delete actions, not browsing or cart activity.

### Schema changes

There is no Alembic. On startup, `main.py` runs `Base.metadata.create_all` and then the idempotent statements in `app/migrations.py` (columns, indexes, constraints, one-time backfills). New schema changes must be added there as idempotent SQL that is safe to run on every boot.

Seed data for a fresh database: `python -m app.seed` (4 pages, 45 categories, 218 products).

## 5. Backend API

The API has 13 routers under `backend/app/routers/`, one per business area, all under `/api/`. Interactive docs are at `/api/docs`; health check at `/api/health`.

| Router | Prefix | Access | Covers |
|---|---|---|---|
| `auth` | `/api/auth` | Public / signed in | Staff password login, customer OTP request/verify, token refresh, logout, profile |
| `catalog` | `/api/catalog` | Public | Pages, offers, a page's full catalog bundle, product detail, reviews |

| Router | Prefix | Access | Covers |
|---|---|---|---|
| `cart`, `wishlist` | `/api/cart`, `/api/wishlist` | Customer | Sync browser cart/wishlist with the server |
| `addresses` | `/api/addresses` | Customer | Saved delivery addresses |
| `orders` | `/api/orders` | Customer | Checkout, confirm COD, my orders, retry payment, cancel, address change |
| `payments` | `/api/payments` | Customer / Razorpay | Verify payment signature, Razorpay webhook, mock pay, refund |
| `uploads` | `/api/uploads` | Customer | Artwork upload to R2 |
| `bookings` | `/api/bookings` | Customer | Create a booking, my bookings |
| `notifications` | `/api/notifications` | Customer | List and mark read |
| `contact` | `/api/contact` | Public | Contact form |
| `admin` | `/api/admin` | Staff (some owner-only) | All dashboard operations, CSV reports, audit log |
| `analytics` | `/api/analytics` | Staff / owner | Summary, orders, bookings, products, logins, actions (see `docs/API_ANALYTICS.md`) |

## Authentication

- **Customers** sign in with a 6-digit email OTP. Codes expire and have a capped number of attempts.
- **Staff and owner** sign in with email and password at `/api/auth/login`.
- Both get a JWT access token (12 hours by default) and a refresh token (14 days) stored in `refresh_tokens` so it can be revoked on logout.
- Blocking an email takes effect on the next request, not just the next login.

## Roles and audit

- `deps.py` provides `get_current_user`, `require_staff` (staff or owner) and `require_owner`.

- Any route exposing personal data from `audit_log` (logins, IPs) must use `require_owner`.
- Every staff change and auth event must call `deps.audit(db, user, action, entity, entity_id, detail, request)`.

### Error handling

- A malformed UUID returns 400, not 500.
- Database pool exhaustion or a pooler refusal returns 503 with `Retry-After: 5`.
- Responses over 1 KB are gzipped; every response carries `X-Process-Time-Ms`.

## 6. External integrations

The backend depends on six external services; each account must be transferred or re-created for the new team (see section 11).

| Service | Used for | Code | Config | If not configured |
|---|---|---|---|---|
| Supabase PostgreSQL | All application data (25 tables) | `database.py` | `DATABASE_URL`, `DB_POOL_*` | App will not start |
| Cloudflare R2 | Product images, media library, customer artwork | `services/storage.py`, `services/media.py` | `R2_*` | Uploads stay inline as base64; checkout still works |
| Cloudinary (legacy) | Former image host; old URLs may remain | `services/storage.py` | `CLOUDINARY_*` | Optional |
| Razorpay | Online payments, signature check, webhooks, refunds | `services/razorpay_service.py`, `routers/payments.py` | `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET` | Mock mode, no real charges |
| SMTP email | Customer login OTP codes | `services/email_service.py` | `SMTP_*` | OTP login fails unless `DEBUG_OTP=true` (code then only logged on the server) |
| Cloudflare tunnel | Public HTTPS URL for the backend | `server-config/cloudflared-studio.service` | None (quick tunnel) | Site cannot reach the API |

### Supabase connection limits

The Supabase session pooler allows 15 clients for this project. The app pool is capped at 10 (3 + 7 overflow) to leave room for deploys, scripts and local development. Raising it risks `EMAXCONNSESSION` errors during checkout.

### Razorpay

Payments are confirmed two ways: the browser posts the signature to `/api/payments/verify`, and Razorpay calls `/api/payments/webhook`. Webhook events are de-duplicated in `webhook_events`. The webhook URL in the Razorpay dashboard must be updated whenever the backend URL changes.

### MCP analytics server

`mcp_server/` exposes read-only business analytics to AI clients such as Claude and Cursor. It holds no database credentials: it signs in to the backend as one staff or owner account and calls `/api/analytics/*`. Each user gets a personal API key in `MCP_API_KEYS`. See `docs/MCP.md` and `mcp_server/README.md`.

## 7. Security and configuration

All secrets live in `.env` files on the server and on developer machines, never in Git. The repository is **public**, so anything committed is visible to everyone.

### Environment variables ( `backend/.env` )

Values below are what production uses today. Secret values are not written here: they are handed over through a password manager (section 11). Variables not set fall back to the code default in `backend/app/config.py`.

| Variable | Production value | Secret | Purpose |
|---|---|---|---|
| `DATABASE_URL` | Supabase session-pooler connection string | Yes | Postgres connection; app will not start without it |
| `SECRET_KEY` | Long random string | Yes | Signs JWTs; changing it signs everyone out |
| `ACCESS_TOKEN_MINUTES` | Set in `.env` (code default 720) | No | Access token lifetime |

| Variable | Production value | Secret | Purpose |
|---|---|---|---|
| `REFRESH_TOKEN_DAYS` | Not set (default 14) | No | Refresh token lifetime |
| `CORS_ORIGINS` | `http://localhost:5500,http://localhost:5501,http://127.0.0.1:5500,http://127.0.0.1:5501,https://theaishastra.github.io` | No | Origins allowed to call the API |
| `SUPABASE_URL` | `https://xdjrtgfynnkmslpanmpb.supabase.co` | No | Not used by the app yet |
| `SUPABASE_SECRET_KEY` | Supabase service key | Yes | Not used by the app yet |
| `ADMIN_EMAIL` | `owner@saikumarstudio.in` | No | Owner account created by the seed |
| `ADMIN_PASSWORD` | Owner password | Yes | Change from the code default |
| `ADMIN_NAME` | `Studio Owner` | No | Owner display name |
| `CLOUDINARY_CLOUD_NAME` | `ismg8jfl` | No | Legacy image host |
| `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` | Cloudinary credentials | Yes | Legacy image host |
| `CLOUDINARY_FOLDER` | `sai_kumar_studio` | No | Legacy folder name |
| `R2_ACCOUNT_ID`, `R2_ACCESS_KEY`, `R2_SECRET_KEY` | Cloudflare R2 API token | Yes | Image storage |
| `R2_BUCKET` | `sai-kumar-data` | No | R2 bucket name |
| `R2_PUBLIC_BASE_URL` | `https://pub-0f96bbc0f4a649b7b396578fc5db875b.r2.dev` | No | Public URL for images, no trailing slash |

| Variable | Production value | Secret | Purpose |
|---|---|---|---|
| `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET` | Razorpay API keys | Yes | Payments; placeholder values switch on mock mode |
| `RAZORPAY_WEBHOOK_SECRET` | Razorpay webhook secret | Yes | Verifies webhook calls |
| `SMTP_HOST` | `smtp.gmail.com` | No | OTP email server |
| `SMTP_PORT` | `587` | No | |
| `SMTP_USER`, `SMTP_FROM_EMAIL` | `theaishastra@gmail.com` | No | Sending mailbox |
| `SMTP_PASSWORD` | Gmail app password | Yes | |
| `SMTP_FROM_NAME` | `Saikumar Digital Lab & Studio` | No | Sender name on OTP emails |
| `SMTP_USE_TLS` | `true` | No | |
| `DEBUG_OTP` | `false` | No | Must stay `false` in production |
| `DB_POOL_SIZE`, `DB_MAX_OVERFLOW`, `DB_POOL_TIMEOUT_SECONDS` | Not set (defaults 3, 7, 20) | No | Keep pool + overflow under the pooler's 15-client limit |

Template for `backend/.env`:

```
DATABASE_URL=<from password manager>
SECRET_KEY=<from password manager>
ACCESS_TOKEN_MINUTES=<copy from the server's .env>
CORS_ORIGINS=http://localhost:5500,http://localhost:5501,http://127.0.0.1:550
0,http://127.0.0.1:5501,https://theaishastra.github.io
SUPABASE_URL=https://xdjrtgfynnkmslpanmpb.supabase.co
SUPABASE_SECRET_KEY=<from password manager>
ADMIN_EMAIL=owner@saikumarstudio.in
ADMIN_PASSWORD=<from password manager>
```

```
ADMIN_NAME=Studio Owner
CLOUDINARY_CLOUD_NAME=ismg8jfl
CLOUDINARY_API_KEY=<from password manager>
CLOUDINARY_API_SECRET=<from password manager>
CLOUDINARY_FOLDER=sai_kumar_studio
R2_ACCOUNT_ID=<from password manager>
R2_ACCESS_KEY=<from password manager>
R2_SECRET_KEY=<from password manager>
R2_BUCKET=sai-kumar-data
R2_PUBLIC_BASE_URL=https://pub-0f96bbc0f4a649b7b396578fc5db875b.r2.dev
RAZORPAY_KEY_ID=<from password manager>
RAZORPAY_KEY_SECRET=<from password manager>
RAZORPAY_WEBHOOK_SECRET=<from password manager>
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=theaishastra@gmail.com
SMTP_PASSWORD=<from password manager>
SMTP_FROM_EMAIL=theaishastra@gmail.com
SMTP_FROM_NAME=Saikumar Digital Lab & Studio
SMTP_USE_TLS=true
DEBUG_OTP=false
```

Template for `mcp_server/.env` (same as `mcp_server/.env.example`). `STUDIO_EMAIL` is the account the MCP server signs in with; `MCP_API_KEYS` is a comma-separated list, one key per person.

```
STUDIO_API_URL=http://127.0.0.1:8000
STUDIO_EMAIL=owner@saikumarstudio.in
STUDIO_PASSWORD=<from password manager>
MCP_API_KEYS=<one key per person, comma-separated>
MCP_HOST=127.0.0.1
MCP_PORT=8100
MCP_ALLOWED_HOSTS=
```

## Never commit

`.gitignore` already excludes `.env`, `backups/`, `backend/backups/` (database dumps with customer data and password hashes), `guides/` (private notes with server IPs and logins) and image backup zips.

### Access control

- Server access is by SSH key (`.pem`) to the `ubuntu` user. The backend binds to `127.0.0.1` only; the public paths are the tunnel and nginx on port 80.
- Staff accounts are created and deactivated by the owner in the admin dashboard.
- Customer emails can be blocked from the dashboard; the block applies immediately.
- MCP API keys give access to customer emails and login IPs; issue one per person and revoke by removing it and restarting `studio-mcp`.

### Exposure found during handover

The committed files `docs/VM_REDEPLOY_GUIDE.txt`, `docs/VM_STOP_START_GUIDE.txt`, `backend/README.md` and `.vscode/settings.json` contain an old EC2 public IP and the local `.pem` key path. The key itself is not in the repo. These should be replaced with placeholders, and the SSH key rotated as part of handover.

## 8. Deployment and operations

The storefront deploys by pushing to GitHub (GitHub Pages at `theaishastra.github.io`); the backend and admin deploy by copying a tarball to the EC2 VM over SSH. The VM has no Git checkout, so pushing to GitHub alone does not update the backend.

### What runs on the VM

| systemd unit | Runs | Port | Unit file |
|---|---|---|---|
| `studio-backend` | uvicorn `app.main:app`, single worker, from `/home/ubuntu/app/backend` | 127.0.0.1:8000 | `server-config/studio-backend.service` |
| `cloudflared-studio` | Cloudflare quick tunnel to the backend | none (outbound) | `server-config/cloudflared-studio.service` |
| `studio-mcp` | MCP analytics server over HTTP | 127.0.0.1:8100 | `server-config/studio-mcp.service` |

| systemd unit | Runs | Port | Unit file |
|---|---|---|---|
| `nginx` | Reverse proxy for `/api`, `/admin`, `/media`, `/mcp` | 80 | `server-config/nginx-studio.conf` |

## Deploy backend and admin changes

1. Commit and push to GitHub.
2. From the repo root in Git Bash, package `backend` and `admin`, excluding `venv` and caches. Exclude `backend/.env` too unless you mean to overwrite the server's copy.
3. `scp` the archive to the VM and extract it into `/home/ubuntu/app`.
4. On the VM: `./venv/bin/pip install -r requirements.txt`, then `sudo systemctl restart studio-backend`.
5. Wait 30–40 seconds, then check `/api/health` through the tunnel URL.

Full commands are in `docs/VM_REDEPLOY_GUIDE.txt`.

## Deploy storefront changes

Push to GitHub; Pages serves the root HTML/JS. When a JS or CSS file changes, bump its `?v=` cache version in the HTML so browsers load the new file.

## Operational rules learned the hard way

- **Single worker only.** `--workers 2` hangs at startup on this instance size.
- **The tunnel URL changes** whenever `cloudflared-studio` restarts (crash, reboot, manual restart). Then update `window.SAI_API_BASE` in `js/shared/api-config.js`, push, and update the Razorpay webhook URL. Find the current URL with `grep trycloudflare /home/ubuntu/cloudflared.log | tail -1` on the VM.
- Keep `Wants=studio-backend.service` in the cloudflared unit. `Requires=` restarts the tunnel on every backend restart and broke the live site on 2026-10-06 and 2026-10-10.
- **The public IP changes** when the EC2 instance is stopped and started. Attaching an Elastic IP avoids this.
- `CORS_ORIGINS` on the VM must include `https://theaishastra.github.io` and any new frontend origin.

### Logs and status

- Backend logs: `sudo journalctl -u studio-backend -f`
- Status: `sudo systemctl status studio-backend cloudflared-studio studio-mcp nginx --no-pager`
- Tunnel log: `/home/ubuntu/cloudflared.log`

### Backups

Supabase free tier has no reliable automatic backup. `backend/scripts/backup_database.py` takes a read-only dump of all 25 tables (schema SQL, data SQL, JSON, CSV, manifest); `restore_database.py` restores it. Typical run: `python scripts/backup_database.py --out "D:\studio-backups" --zip --keep 30`. Backups contain customer data and must never be committed. See `docs/DATABASE_BACKUP_GUIDE.txt`.

Open question: is a backup run on a schedule today, or only manually?

## 9. Local development and testing

A developer needs Python 3.12, Git, and a `backend/.env` pointing at a **non-production** database. The storefront has no build step.

### Run it

1. `cd backend`, then `py -3.12 -m venv venv` and `.\venv\Scripts\Activate.ps1`.
2. `pip install -r requirements.txt`.
3. Create `backend/.env` (variables in section 7). Use a separate Supabase project or a local Postgres, not the live one.
4. `python -m app.seed` on an empty database.
5. `uvicorn app.main:app --reload --port 8000`. API docs: `http://localhost:8000/api/docs`.
6. From the repo root, `python -m http.server 5500` and open `http://localhost:5500`. On localhost, `api-config.js` points the site at `http://127.0.0.1:8000` automatically.
7. Admin dashboard: `http://localhost:8000/admin/`.

The MCP server has its own venv and setup in `mcp_server/README.md`.

### Testing

There is no automated test suite or CI yet. Current practice:

- For endpoint checks, mount one router in a bare `FastAPI()` app with an in-memory SQLite engine and override `get_db` and `get_current_user`.
- `backend/scripts/test_e2e_*.py` are end-to-end scripts for specific fixes (concurrency, customer deletion, option pictures).
- `backend/test-api.ps1` pings key endpoints on a running server.
- **Never** point tests or scripts at the production `DATABASE_URL`. Scripts such as `delete_all_orders.py` are destructive.

### Useful scripts ( `backend/scripts/` )

| Script | Does |
|---|---|
| `backup_database.py` / `restore_database.py` | Back up and restore all tables |
| `set_admin_password.py` | Reset a staff or owner password |
| `rotate_secrets.py` | Rotate application secrets |
| `migrate_supabase.py` | Move data between Supabase projects |
| `delete_all_orders.py` | Wipes orders. Destructive, test databases only |

### Code conventions

- Comments explain *why* (incidents, constraints), not what.
- Schema changes go in `app/migrations.py` as idempotent statements.
- "Today" in analytics is the Indian Standard Time calendar day.
- `unused/` holds retired scripts and data for reference only.

## 10. Known issues and recommendations

The biggest operational risk is the free Cloudflare quick tunnel: any restart changes the API URL and takes the live site offline until the frontend is updated. A full security and architecture audit from 2026-09-19 is in `docs/AUDIT_REPORT.md`; several of its items have

since been fixed, but no per-item status is kept.

| Priority | Issue | Impact | Recommendation |
|---|---|---|---|
| High | Quick tunnel URL changes on every `cloudflared` restart | Site cannot reach the API; manual frontend edit and push needed | Move to a named Cloudflare tunnel or a custom domain with TLS |
| High | Old EC2 IP and `.pem` key path committed in the public repo | Exposes server details | Replace with placeholders, rotate the SSH key |
| High | Audit findings not tracked item by item | Unclear which risks remain (pricing, XSS, refunds, stock races) | Re-verify each item in `docs/AUDIT_REPORT.md` and record its status |
| Medium | No automated tests or CI | Regressions reach production unnoticed | Add pytest suite on SQLite plus a GitHub Actions run |
| Medium | Manual tarball deploy, which also overwrites `backend/.env` by default | Wrong keys can go live | Script the deploy and always exclude `.env` |
| Medium | Public IP changes on instance stop/start | SSH and nginx route break | Attach an Elastic IP |
| Medium | Single VM, single worker, in-process sweeps and cache | No redundancy; cannot scale out | Acceptable at current traffic; needs shared scheduler/cache before a second instance |
| Medium | Migrations run at boot with no history or rollback | Risky schema changes | Adopt Alembic when the schema next changes materially |
| Medium | Backups run manually | Data loss if Supabase project is lost | Schedule `backup_database.py` and store copies off-machine |
| Low | Auth tokens kept in `localStorage` | Token theft if an XSS bug appears | CSP is in place; consider httpOnly cookies later |

| Priority | Issue | Impact | Recommendation |
|---|---|---|---|
| Low | Some docs are outdated (`backend/README.md` status claims, Cloudinary references, old IPs) | Confuses new developers | Refresh docs; this document supersedes them where they differ |

## 11. Handover checklist

Handover is complete when the new team can deploy, restart and restore the system on their own, using accounts they control. Hand credentials over through a password manager or in person, never by email, chat or Git.

### Accounts and access to transfer

- [ ] GitHub repository `theaishastra/studio-backed` (admin access, and GitHub Pages settings)
- [ ] AWS account or IAM user with EC2 access to the studio instance
- [ ] SSH key for the VM; generate a new key pair for the new team and remove the old one
- [ ] Supabase project (owner role) and the database password
- [ ] Cloudflare account (R2 bucket and its API token; tunnel)
- [ ] Razorpay dashboard (API keys, webhook secret, webhook URL)
- [ ] SMTP mailbox used for OTP emails
- [ ] Cloudinary account (legacy, if still holding images)
- [ ] Owner login for the admin dashboard; create named staff accounts for each new team member
- [ ] Copies of `backend/.env` and `mcp_server/.env` from the VM
- [ ] Private `guides/` notes and the latest database backup, handed over offline
- [ ] Issue new MCP API keys; remove keys belonging to people leaving

### Knowledge transfer sessions

- [ ] Walk through this document and the architecture diagram
- [ ] Live deploy of a small backend change following section 8
- [ ] Restart `cloudflared-studio` together and update the tunnel URL end to end
- [ ] Take a backup and restore it into a test Supabase project

- [ ] Tour of the admin dashboard with the studio owner: orders, bookings, catalog, refunds

## First-week tasks for the new team

- [ ] Set up local development against a non-production database (section 9)
- [ ] Scrub IPs and key paths from committed docs and rotate the SSH key (section 10)
- [ ] Rotate `SECRET_KEY`, database password and third-party keys after access changes hands
- [ ] Decide on a stable API domain to replace the quick tunnel
- [ ] Confirm backups run on a schedule

## Related documents in the repo

- `CLAUDE.md`: layout and conventions summary
- `docs/SETUP.md`: local setup
- `docs/VM_REDEPLOY_GUIDE.txt`, `docs/VM_STOP_START_GUIDE.txt`, `docs/AWS_VM_Deployment_Runbook.pdf`: server operations
- `docs/DATABASE_BACKUP_GUIDE.txt`, `docs/PROJECT_MIGRATION_GUIDE.txt`: data
- `docs/API_ANALYTICS.md`, `docs/MCP.md`, `mcp_server/README.md`: analytics and MCP
- `docs/AUDIT_REPORT.md`: security and architecture audit